

ZESPÓŁ SZKÓŁ ELEKTRYCZNO-MECHANICZNYCH

Pracownia testowania i dokumentowania aplikacji

Testowanie aplikacji

Autor: Dawid Łazarz Klasa: 4P Prowadzący: mgr inż. Kamil Wojnarowski

NOWY SĄCZ 2026 r.

Spis treści

- Wstęp | 4
 - Testy jednostkowe | 5
 - 1.1 Implementacja programu | 5
 - 1.2 Implementacja testów | 5
 - 1.3 Git | 5
- Podsumowanie | 9
- Bibliografia | 10

Wstęp

Testowanie oprogramowania jest jednym z kluczowych elementów procesu wytwarzania aplikacji, którego głównym celem jest weryfikacja poprawności działania systemu oraz wykrycie ewentualnych błędów na jak najwcześniejszym etapie rozwoju.

Podstawowym zagadnieniem w inżynierii oprogramowania jest zapewnienie jakości (ang. Quality Assurance). Testowanie pozwala upewnić się, że kod spełnia wymagania funkcjonalne oraz biznesowe. W procesie testowym wyróżniamy różne poziomy testów, takie jak testy jednostkowe, integracyjne, systemowe oraz akceptacyjne. Wczesne wykrywanie usterek drastycznie redukuje koszty naprawy błędów oraz zwiększa bezpieczeństwo i stabilność tworzonych rozwiązań.

1. Testy jednostkowe

Testy jednostkowe (ang. Unit Tests) polegają na izolowanym sprawdzaniu najmniejszych fragmentów kodu, takich jak pojedyncze metody lub klasy, w celu weryfikacji ich prawidłowego działania. Wykonujemy je na bieżąco podczas pisania kodu (często zgodnie z metodologią Test-Driven Development - TDD).

Przykład opisowy: Wyobraźmy sobie kalkulator z funkcją dzielenia. Test jednostkowy podaje do metody dwie liczby (np. 10 i 2) i sprawdza, czy wynik wynosi dokładnie 5. Osobny test weryfikuje przypadek skrajny - przekazanie zera jako dzielnika i sprawdzanie, czy system poprawnie zgłosi błąd dzielenia przez zero.

Narzędzie xUnit: W projekcie wykorzystano xUnit - nowoczesny, otwarty framework testowy dla platformy .NET. Pozwala on na łatwe definiowanie przypadków testowych za pomocą adnotacji (np. [Fact]) oraz weryfikację wyników przy użyciu asercji (np. Assert.Equal, Assert.Throws).

1.1 Implementacja programu

W ramach zadania zaimplementowano w języku C# program realizujący algorytm sortowania bąbelkowego (Bubble Sort) wraz z walidacją danych wejściowych.

Kod programu (Program.cs):

```
C#  
1. using System;  
2.  
3. public class Program  
4. {  
5.     public static void Main(string[] args)
```

```
6.  {
7.    int[] numbers = { 64, 34, 25, 12, 22, 11, 90 };
8.
9.    Console.WriteLine("Tablica przed sortowaniem:");
10.   PrintArray(numbers);
11.
12.   BubbleSort(numbers);
13.
14.   Console.WriteLine("Tablica po sortowaniu:");
15.   PrintArray(numbers);
16. }
17.
18. public static void BubbleSort(int[] array)
19. {
20.     // WARUNEK BEZPIECZEŃSTWA: Jeśli tablica jest null lub pusta, przerywamy
21.     if (array == null || array.Length == 0) return;
22.
23.     // WALIDACJA (Test negatywny): Ograniczenie rozmiaru tablicy do 1000 elementów
24.     if (array.Length > 1000)
25.     {
26.         throw new ArgumentException("Tablica jest zbyt duża do sortowania
bąbelkowego!");
27.     }
28.
29.     int n = array.Length;
30.
31.     for (int i = 0; i < n - 1; i++)
32.     {
33.         for (int j = 0; j < n - i - 1; j++)
34.         {
35.             if (array[j] > array[j + 1])
36.             {
37.                 int temp = array[j];
38.                 array[j] = array[j + 1];
39.                 array[j + 1] = temp;
40.             }
41.         }
42.     }
43. }
44.
45. public static void PrintArray(int[] array)
46. {
47.     if (array == null) return;
```

```

48.     Console.WriteLine(string.Join(", ", array));
49. }
50. }

```

Opis działania kodu: Metoda BubbleSort przyjmuje jako parametr tablicę liczb całkowitych. Na początku weryfikuje warunki brzegowe: brak tablicy (null) lub pustą tablicę. Ponadto zawiera regułę biznesową ograniczającą wielkość tablicy do 1000 elementów - przekroczenie tego limitu skutkuje rzuceniem wyjątku ArgumentException. Sam algorytm wykorzystuje dwie zagnieżdżone pętle for, które porównują sąsiadujące elementy i w razie potrzeby zamieniają je miejscami za pomocą zmiennej pomocniczej temp.

1.2 Implementacja testów

Zostały zaimplementowane 4 testy jednostkowe przy użyciu biblioteki xUnit (3 testy pozytywne oraz 1 test negatywny).

Kod testów (sortBobelkoweTest.cs):

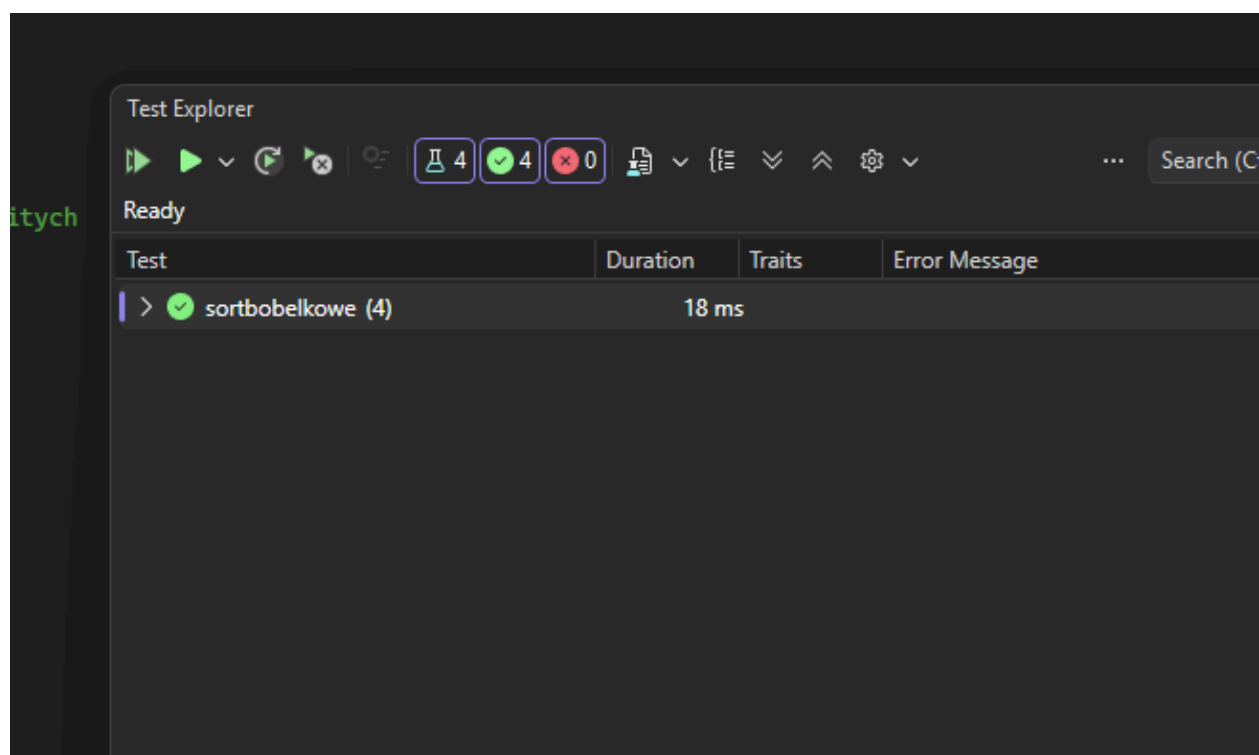
```

C#
51. using System;
52. using Xunit;
53.
54. public class BubbleSortTests
55. {
56.     // --- TESTY POZYTYWNE ---
57.
58.     [Fact]
59.     public void Test_ValidArray_SortsCorrectly()
60.     {
61.         // Sprawdzamy czy metoda poprawnie sortuje zwykłą tablicę
62.         int[] input = { 3, 1, 2 };
63.         Program.BubbleSort(input);
64.         Assert.Equal(new[] { 1, 2, 3 }, input);
65.     }
66.
67.     [Fact]
68.     public void Test_NullArray_HandlesGracefully()
69.     {
70.         // Sprawdzamy czy program nie wywali się przy wartości null
71.         int[]? input = null;
72.         Program.BubbleSort(input);
73.         Assert.Null(input);

```

```
74. }
75.
76. [Fact]
77. public void Test_EmptyArray_HandlesGracefully()
78. {
79.     // Sprawdzamy czy program poprawnie obsługuje pustą tablicę
80.     int[] input = { };
81.     Program.BubbleSort(input);
82.     Assert.Empty(input);
83. }
84.
85. // --- TEST NEGATYWNY ---
86.
87. [Fact]
88. public void Test_TooLargeArray_ThrowsArgumentException()
89. {
90.     // Podajemy za dużą tablicę (ponad 1000 elementów)
91.     int[] tooLargeArray = new int[1001];
92.
93.     // Sprawdzamy czy algorytm wyrzuci oczekiwany błąd (ArgumentException)
94.     Assert.Throws<ArgumentException>(() => Program.BubbleSort(tooLargeArray));
95. }
96. }
```

Wyniki wykonania testów: Wszystkie testy zostały uruchomione w oknie Test Explorer środowiska Visual Studio i zakończyły się sukcesem (4/4 passed).



1.3 Git

W trakcie tworzenia programu oraz zestawu testów prowadzona była kontrola wersji przy użyciu systemu Git. Wykonano wymagane commity rejestrujące postęp prac nad kodem źródłowym i testami.

```
UCRT64\Users\dawid\Desktop\projekt
$ git log
commit 7f9fd63dc8bde33a8186d78641cf044af3e9a (HEAD -> main, origin/main)
Author: S1ejerr <dawid14463@gmail.com>
Date: Mon Oct 5 20:08:07 2026 +0200

    Dodanie szczegółowych komentarzy do kodu i funkcji pomocniczych

commit 979411c82d2dbca05eb24a31a43f7a1a06118
Author: S1ejerr <dawid14463@gmail.com>
Date: Mon Oct 5 20:00:35 2026 +0200

    Dodanie algorytmu sortowania bąbelkowego

commit 246872f0f542ccf0b47263f9584e3489dfe9d0
Author: S1ejerr <dawid14463@gmail.com>
Date: Mon Oct 5 19:54:07 2026 +0200

    Implementacja funkcji pomocniczej Swap do zamiany elementów

commit 2f38f32f69b9c8f10b61933449b67c4b8a7cc6
Author: S1ejerr <dawid14463@gmail.com>
Date: Mon Oct 5 19:41:52 2026 +0200

    Dodanie struktury programu i metody do wyświetlania tablicy

commit c589c7b0c83bae2825773ceb0d715286962ea (origin/testy, testy)
Author: S1ejerr <dawid14463@gmail.com>
Date: Mon Oct 5 18:05:13 2026 +0200

    opis zmian

commit 15ed7a5c9b4d4e9321d0311d32d64dbed8518
Author: S1ejerr <dawid14463@gmail.com>
Date: Mon Oct 5 18:21:14 2026 +0200

    Dodanie zmiennej globalnej

commit b0a67d0f4d3205a0277f6e84df104013926c2ad9
Author: S1ejerr <dawid14463@gmail.com>
Date: Sun Oct 4 20:16:34 2026 +0200

    Inicjacja repozytorium
```

Podsumowanie

W ramach przeprowadzonego zadania pomyślnie zaimplementowano algorytm sortowania bąbelkowego w języku C# oraz stworzono zestaw automatycznych testów jednostkowych w frameworku xUnit. Przeprowadzone testy (3 pozytywne oraz 1 negatywny) potwierdziły poprawność działania programu zarówno dla standardowych danych, przypadków brzegowych, jak i sytuacji zgłaszania błędów walidacji.

Bibliografia

1. Dokumentacja języka C# i .NET: <https://learn.microsoft.com/dotnet/csharp/>
2. Dokumentacja frameworka xUnit: <https://xunit.net/docs/getting-started/netcore/cmdline>
3. Dokumentacja systemu kontroli wersji Git: <https://git-scm.com/doc>